

COURSE

Основы программирования JS



Инструментарий и основы программирования на JavaScript

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции ***3***

Теоретическая часть ***3***

Обзор сервисов и IDE для разработки на Jscript. **3**

Библиотека Node.js. **6**

Установка Node.js 6

Начало работы (режим REPL) 7

Модульность приложений 8

Пакетный менеджер NPM. **10**

Добавление зависимостей 10

Добавление зависимостей для разработки 11

NPM-скрипты 11

Ограничение области видимости и приватные пакеты 12

Редактор MS Visual Code. **12**

Настройка отладки скриптов VS Code через Node.js 13

IDE JetBrains WebStorm. **15**

Литература и другие источники ***17***

Вопросы для самопроверки ***17***

ПЛАН ЛЕКЦИИ

1. Обзор сервисов и IDE для разработки на Jscript.
2. Библиотека Node.js.
3. Пакетный менеджер NPM.
4. Редактор MS Visual Code.
5. IDE JetBrains WebStorm.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

ОБЗОР СЕРВИСОВ И IDE ДЛЯ РАЗРАБОТКИ НА JSCRIPT.

Рассмотрим полезные онлайн сервисы, которые могут значительно упростить разработку программного кода на JavaScript. Можно отметить сразу, что большинство онлайн-сервисов не являются русскоязычными и, чтобы воспользоваться данными сервисами достаточно обладать минимальными знаниями английского языка.

Сервис JSBeautifier (http://jsbeautifier.org/)

Как известно, любой код должен быть отформатирован и понятен, но не все любят форматировать свой код, а кому-то просто лень этим заниматься. Данный сервис сделает форматирование кода JavaScript за вас.

На странице сервиса нужно просто скопировать свой код JavaScript (или HTML) в текстовое поле и запустить форматирование. В итоге вы получаете красивый, понятный, отформатированный код, который радует глаз. В общем, данный сервис будет полезен на конечном этапе формирования описания программного кода или включения его в состав печатных документов.

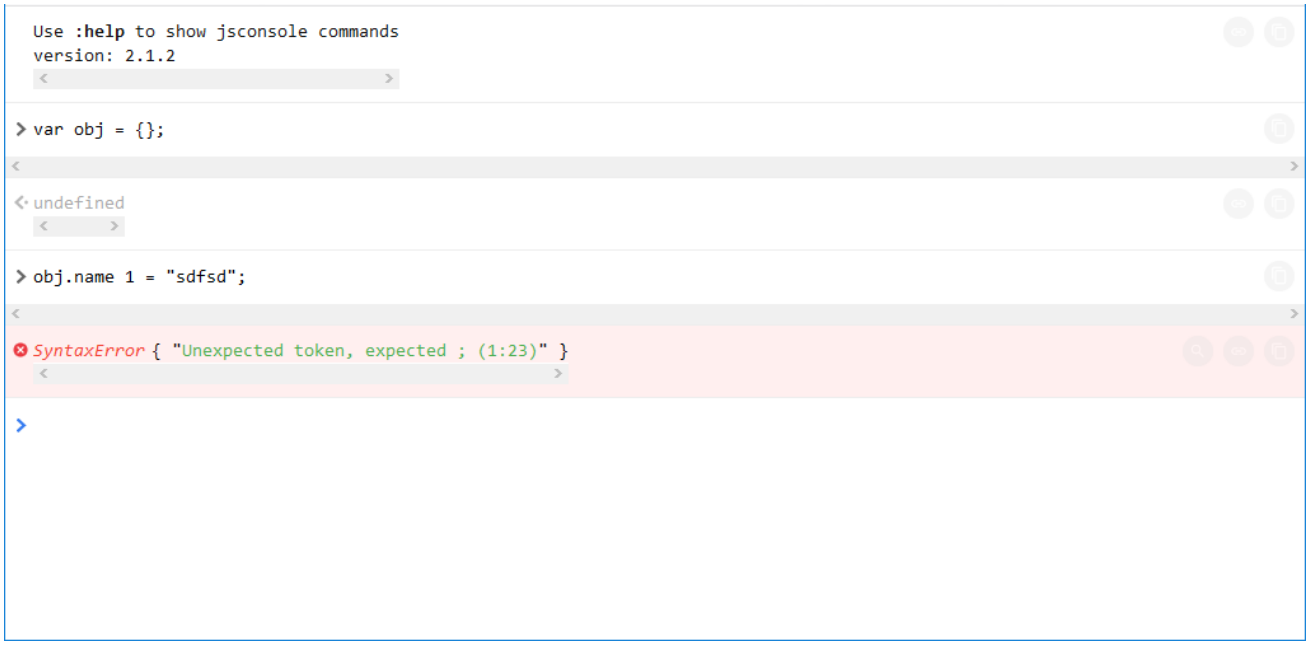


Следует отметить, что альтернативой данному сервису служат мощные редакторы (среды программирования) с установленными библиотеками, поддерживающими последние стандарты JavaScript, например, среда программирования WebStorm. В этой среде разработки также присутствует функциональность по автоматическому форматированию программного кода.

Сервис JSconsole (http://jsconsole.com/)

Отладить программу на JavaScript – не самая легкая задача. А на поиск ошибок в коде программы JavaScript может уйти много времени.

Рассматриваемый онлайн-сервис позволяет искать ошибки и отлаживать JavaScript программы. Пользуйтесь данным сервисом и существенно экономьте время. Нужно сказать, что большинство сервисов исполнения кода JavaScript «проглатывают» критические ошибки, просто не отображая результат. Данный сервис очень напоминает консоль Node.js и исполнение программного кода в REPL режиме.



В сервисе имеется возможность не только исполнять код по одной инструкции в режиме «запрос-ответ», но и загружать на выполнение файл .js с программным кодом. В среду исполнения скриптов погружены наиболее часто используемые библиотеки, список которых можно посмотреть командой **:libraries**.

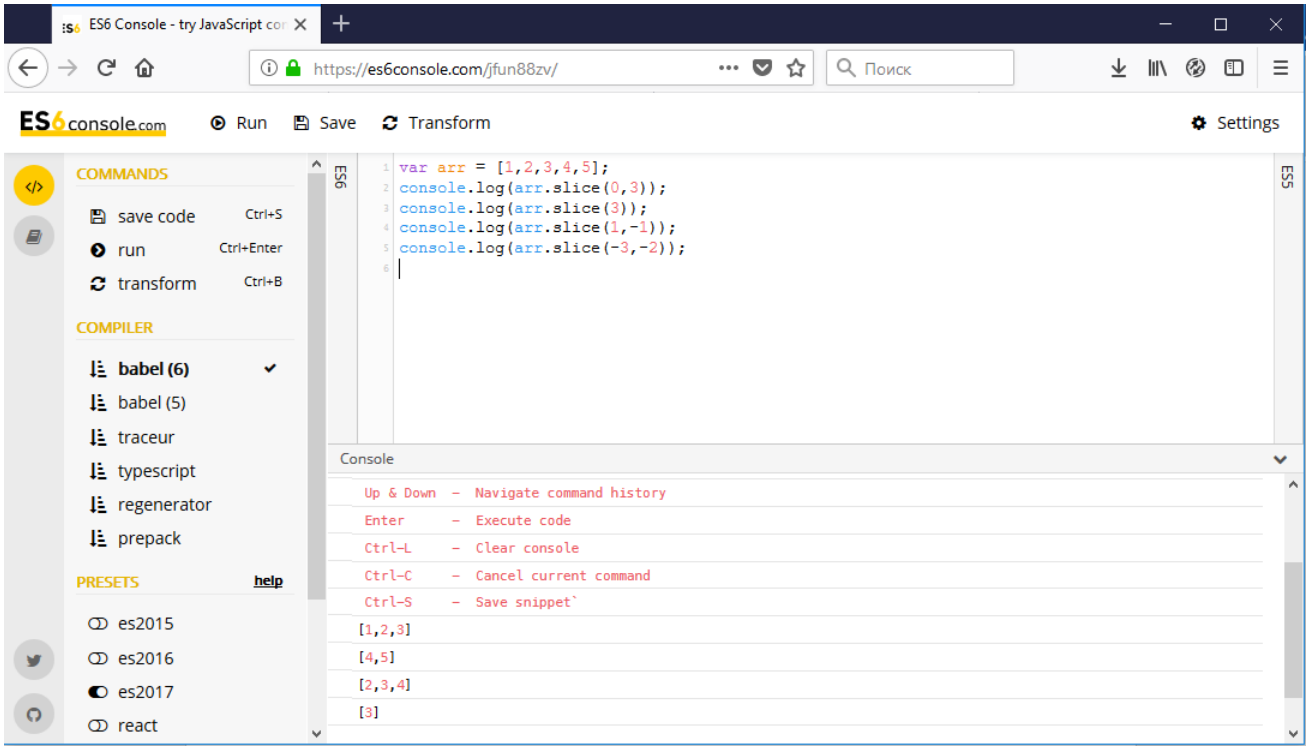
Не следует воспринимать этот сервис как полноценную среду разработки; в нем нет возможности вести привычный процесс отладки с установкой точек останова и просмотра состояния стека. Но если в загружаемых скриптах имеются ошибки, то сервис их отобразит.

Сервис ES6console.com (http://es6console.com)

Следующий онлайн-сервис будет полезен при составлении простых программ на JavaScript. Он работает как простейший редактор с возможностью исполнения многострочного кода. Результат выполнения кода выводится на консоль.

Отличительной особенностью данного сервиса является то, что в нем представлена возможность разработки кода под разными стандартами ESMAScript. При этом данный процесс идет в полуавтоматическом режиме, т.е. программный код просто транслируется под выбранный стандарт и можно посмотреть отличия.

Имеется возможность загрузки кода JavaScript из текстового файла и сохранения результата в файл.

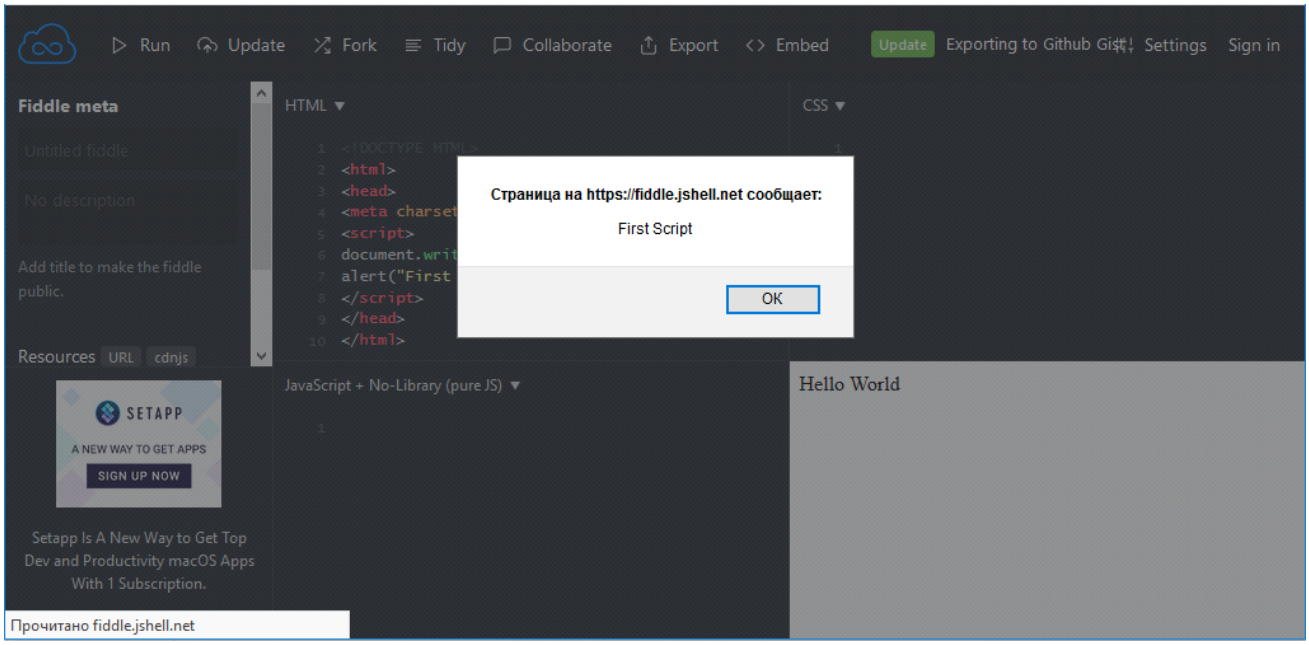


Из минусов, которые удалось подчеркнуть при работе с данным сервисом. Здесь очень слабо представлена обработка ошибок. При наличии некоторых ошибок (например, синтаксических) исполняемая программа может просто «зависнуть», или результат не отобразится на консоли без каких-либо пояснений.

Поэтому данный сервис можно рекомендовать как первоначальный инструментарий для знакомства с языком JavaScript и составления простых скриптов.

Сервис jsFiddle (http://jsfiddle.net/)

Данный сервис является полноценным облачным сервисом с возможностью совместной работы. Построен по принципу одновременной работы над программным кодом через облачный сервис нескольким пользователям, с отображением правок в реальном времени, текстовым и голосовым чатом.



На странице сервиса имеется четыре поля: CSS, HTML, JavaScript и Results. В поле CSS записываются CSS стили, в поле HTML записывается код страниц, а поле JavaScript записываем код JavaScript. При запуске всего на выполнение в поле Results появится результат.

Хочется отметить то, что не встречается в ранее рассмотренных сервисах – это возможность работы с пользовательскими диалоговыми окнами, типа alert или confirm. При этом по работе складывается впечатление, что программный код выполняется непосредственно на браузере (оно так и происходит) просто в него добавлен удобный интерфейс.

Библиотека Node.js.

Node.js — это среда выполнения JavaScript, построенная на JavaScript-движке V8 из Chrome. В основе Node.js лежит событийно-управляемая модель с неблокирующими операциями I/O, что делает её легкой и эффективной.

Другими словами, Node.js предлагает вам возможность писать невероятно производительный серверный код с использованием JavaScript. Как говорится в официальном описании: Node.js — это среда выполнения, использующая тот же JavaScript-движок V8, который вы можете найти в браузере Google Chrome. Но этого недостаточно для успеха Node.js. В Node.js используется библиотека libuv - это кроссплатформенная библиотека поддержки с акцентом на асинхронный ввод-вывод.

С точки зрения разработчика, Node.js однопоточна, но в библиотека libuv используются треды, события файловой системы, реализуется цикл событий, который включает в себя тред-пулинг и так далее. Все это делает библиотеку Node.js удобным инструментом для реализации многопоточных приложений.

Установка Node.js

Последнюю версию Node.js вы можете найти на официальном сайте: https://nodejs.org/en/. На главной странице сайта предлагается скачать последнюю версию продукта и последнюю стабильную версию (так называемую «долгоиграющую» LTS сборку). Запускается инсталлятор или устанавливается пакет. Node.js является кроссплатформенным приложением, поэтому вы можете установить продукт под Windows, OSX или Linux. Интерфейс и функциональные возможности ничем отличаться не будут.

При таком подходе довольно легко начать работу, но, если позже вы захотите добавить в систему больше версий Node.js, лучше сразу начать использовать nvm (node version manager) - диспетчером версий Node.js. После его установки под выбранную платформу вы сможете использовать очень простой CLI API для смены версии Node.js.

Установка различных версий Node.js:

nvm install 8.5

Затем, если вы хотите проверить в работе экспериментальную версию:

nvm install 9

Чтобы убедиться, что у вас установлена и запущена Node.js, выполните (перед version ставиться два тире):

node --version

Если все в порядке, последняя команда вернет номер версии текущего активного бинарного файла Node.js.

Если вы работаете над проектом, поддерживающим Node.js v8, вы можете начать использовать эту версию с помощью следующей команды:

nvm use 8

Затем вы можете переключиться на Node.js v9 с помощью той же самой команды:

nvm use 9

С тех пор как был сформирован Node.js Foundation, Node.js имеет план релизов. Это очень похоже на другие проекты Linux Foundation. Это означает, что есть два релиза: стабильный и экспериментальный. В Node.js стабильными версиями с долговременной поддержкой (**LTS**) являются те, которые начинаются с четных чисел (4, 6, 8, …). Экспериментальные версии нумеруются нечетными числами (5, 7, 9, …).

Начало работы (режим REPL)

Чтобы начать работу с Node.js, давайте попробуем её в консоли! Режим работы с Node.js через консоль называется **REPL**. Здесь задействуется модуль repl, представляющий собой реализацию **Real-Eval-Print-Loop (Чтение-оценка-печать-цикл, REPL)**, которая доступна как в виде автономной программы, так и встроенной в другие приложения.

Нижеследующие команды поддерживаются всеми экземплярами REPL:

- **.break** - в процессе ввода многострочного выражения ввод команды .break (или нажатие комбинации клавиш <Ctrl>-C) прервет последующий ввод или обработку этого выражения.
- **.clear** - сбрасывает context REPL до пустого объекта и стирает все многострочные выражения, которые вводятся в данный момент.
- **.exit** – закрывает все стримы I/O, заставляя REPL завершиться.
- **.help** – показывает список специальных команд.
- **.save** – сохраняет текущую сессию REPL в файл > .save ./file/to/save.js.
- **.load** – загружает файл в текущую сессию REPL > .load ./file/to/load.js.
- **.editor** – входит в режим правки (<Ctrl>-D для завершения, <Ctrl-C> для отмены)

Следующие комбинации клавиш в REPL работают так:

- **<Ctrl>-C** – при единоразовом нажатии имеет такой же эффект, как команда .break. При двойном в пустой строке – такой же эффект, как и .exit.
- **<Ctrl>-D** – имеет такой же эффект, как команда .exit.
- **<tab>** - при нажатии в пустой строке отображает глобальные и локальные переменные. При нажатии во время ввода показывает варианты автозавершения.

Итак, чтобы начать работу нужно открыть консоль (терминал) и запустить Node.js, просто набрав node:

$ node

>

Можно попробовать что-то напечатать:

$ node

> console.log('hello from Node.js')

После нажатия Enter вы получите следующее:

> console.log('hello from Node.js')

hello from Node.js

undefined

Режим REPL очень хорош для проверки однострочных конструкций, но с Node.js удобнее работать, запуская на выполнения приложения. Под **приложением** в данном случае понимается программный код, оформленный в виде скрипта .js. Попробуем создать подобное приложение.

Начнем с создания файла index.js. Откройте свою IDE (Atom, Sublime, VS Code или просто блокнот - выбор за вами), создайте новый файл и сохраните его с именем index.js. Затем скопируйте в него следующий фрагмент кода:

```
// index.js
console.log('hello from Node.js')
```

Чтобы запустить этот файл, вы должны снова открыть свой терминал и перейти в каталог, в котором размещён index.js. Как только вы успешно переместитесь в нужное место, запустите файл, используя команду

```
>node index.js
```

Вы увидите, что эта команда будет выдавать тот же результат, что и раньше, выводя строку непосредственно в терминале.

Модульность приложений

Теперь у нас есть файл index.js, поэтому пришло время перейти на следующий уровень! Давайте создадим что-то более сложное, разделив наш исходный код на несколько JavaScript-файлов с целью удобочитаемости и поддерживаемости. Чтобы начать работу, вернитесь в свою IDE и создайте следующую структуру каталогов (с пустыми файлами), но пока не трогайте package.json, его можно сгенерировать автоматически на следующем шаге:

```
├── app
|   ├── calc.js
|   └── index.js
├── index.js
└── package.json
```

Каждый проект Node.js начинается с создания файла **package.json**. Вы можете думать о нем как о **JSON-представлении приложения и его зависимостей**. Он содержит имя вашего приложения, автора (вас) и все зависимости, необходимые для запуска приложения.

Вы можете интерактивно генерировать файл package.json с помощью команды **npm init** в терминале. После запуска команды требуется ввести некоторые данные, например, имя вашего приложения, версию, описание и так далее. Не нужно беспокоиться, просто нажимайте Enter, пока не получите сформированный JSON и вопрос **is it ok?**. Нажмите Enter в последний раз и вот: ваш package.json был автоматически сгенерирован и помещен в папку вашего приложения. Если вы откроете этот файл в своей IDE, он будет очень похож на фрагмент кода ниже.

```
// package.json
{
  "name": "node-hero",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
```

```
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js"
  },
  "author": "", "license": "ISC"
}
```

Хорошей практикой является добавление стартового скрипта в ваш пакет package.json. Как только вы это сделаете, как показано в примере выше, вы можете запустить приложение с помощью команды **npm start**. Это очень удобно при переносе и развертывании приложений.

Теперь давайте вернемся к первому созданному вами файлу под названием index.js. Лучше оставить этот файл очень компактным: только подключение самого приложения (файл index.js из подкаталога /app, созданного ранее). Скопируйте следующий код в свой файл index.js и сохраните:

```
// index.js
require('./app/index')
```

Теперь пришло время приступить к созданию реального приложения. Откройте файл index.js из папки /app, чтобы создать очень простой пример: добавление массива чисел. В этом случае файл index.js будет содержать только числа, которые мы хотим добавить, а логика, требующая вычислений, должна быть помещена в отдельный модуль.

Вставьте этот код в файл index.js в каталоге /app.

```
// app/index.js
const calc = require('./calc')
const numbersToAdd = [
  3,
  4,
  10,
  2
]

const result = calc.sum(numbersToAdd)
console.log(`The result is: ${result}`)
```

Теперь вставьте фактическую бизнес-логику в файл calc.js, который можно найти в той же папке.

```
// app/calc.js
function sum (arr) {
  return arr.reduce(function(a, b) {
    return a + b
  }, 0)
}

module.exports.sum = sum
```

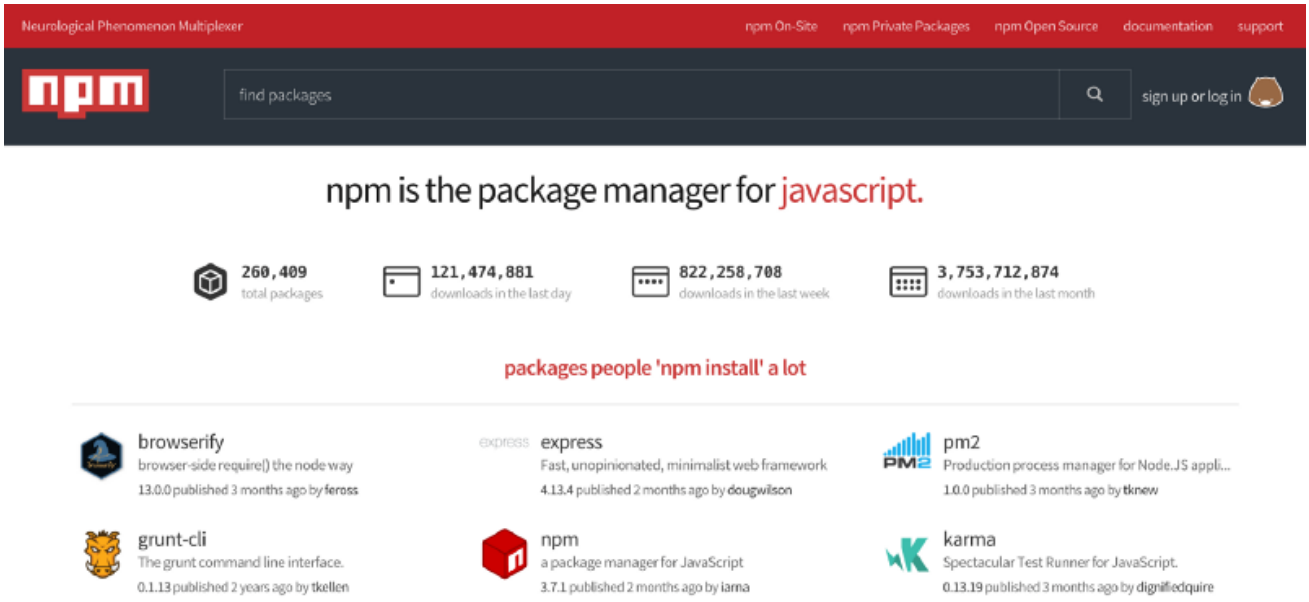
Чтобы проверить, всё ли вы сделали правильно, сохраните эти файлы, откройте терминал и введите **npm start** или **node index.js**. Если вы все сделали правильно, вы получите ответ: **19**. Если что-то пошло не так, внимательно просмотрите лог в консоли и найдите проблему на его основе.

На основе данного примеры была рассмотрена возможность использования модулей при разработке кода и организации модульной структуры приложения Node.js.

Пакетный менеджер NPM.

NPM - это менеджер пакетов, используемый Node.js-приложениями. В нём вы можете найти массу готовых модулей, поэтому вам не нужно «изобретать колесо». Модули создаются разработчиками по всему миру и несут различную функциональность. NPM похож на Maven для Java или Composer для PHP.

Существует два основных интерфейса, с которыми вы будете взаимодействовать: **сайт NPM** и **набор инструментов командной строки (CLI)**. И веб-сайт, и CLI используют один и тот же реестр, чтобы искать и отображать модули. Сайт NPM можно найти по адресу https://npmjs.com. Здесь вы можете зарегистрироваться как новый пользователь или поискать нужные пакеты.



Чтобы запустить CLI, достаточно написать:

>npm

Обратите внимание, что NPM поставляется вместе с бинарным файлом Node.js, поэтому вам не нужно его устанавливать, однако если вы хотите использовать определенную версию NPM, вы можете его обновить. Например, если вы хотите установить NPM версии 4, вы можете сделать это с помощью: npm install npm@4 -g.

Чтобы посмотреть версию установленного NPM, нужно запустить команду:

>npm --version

Добавление зависимостей

В этом разделе вы узнаете, как добавлять рантайм зависимости к вашему приложению. Когда у вас есть файл package.json, вы можете добавить зависимости к вашему приложению. Давайте добавим одну. Попробуйте следующее:

> npm install lodash --save

С помощью этой единственной команды мы достигли двух вещей: во-первых, **lodash** загружен и помещён в папку **node_modules**. Это папка, в которой будут находиться все ваши внешние зависимости. Обычно вы не хотите добавлять её в свою систему управления версиями, поэтому, если вы используете git, обязательно добавьте node_modules в файл .gitignore.

Давайте посмотрим, что происходит в файле package.json. Появилось новое свойство, называемое dependencies:

```
"dependencies": {
  "lodash": "4.6.1"
}
```

Это означает, что lodash версии 4.6.1 теперь установлен и готов к использованию. Обратите внимание, что NPM следует правилам SemVer (http://semver.org/) для версионирования пакетов.

Поскольку lodash готов к использованию, давайте посмотрим, как мы можем им воспользоваться. Вы можете сделать это так же, как и с вашим собственным модулем, но теперь вам не нужно указывать полный путь, достаточно только имени модуля:

```
// index.js
const _ = require('lodash')
_.assign({ 'a': 1 }, { 'b': 2 }, { 'c': 3 });
// → { 'a': 1, 'b': 2, 'c': 3 }
```

Добавление зависимостей для разработки

Когда вы собираете веб-приложения, вам может потребоваться минимизировать ваши JavaScript-файлы, объединить CSS-файлы и так далее. Модули, которые это сделают, будут выполняться только во время создания ресурсов, поэтому работающее приложение не нуждается в них.

Вы можете установить такие скрипты с помощью:

```
> npm install mocha --save-dev
```

После этого в вашем файле package.json появится новый раздел, называемый devDependencies. Все модули, которые вы устанавливаете с помощью --***save-dev***, будут описаны там, а также они будут помещены в тот же самый каталог ***node_modules***.

NPM-скрипты

NPM-скрипты — очень мощная концепция, с их помощью вы можете создавать небольшие утилиты или даже описывать сложные системы сборки. Наиболее распространенными являются скрипты **start** и **test**. С помощью start вы можете описать, как нужно запускать приложение, а test используется для запуска тестов. В вашем package.json они могут выглядеть примерно так:

```
"scripts": {
  "start": "node index.js",
  "test": "mocha test",
  "your-custom-script": "echo npm"
}
```

Что следует здесь отметить:

- **start**: просто описывает начальную точку для запуска вашего приложения, этот скрипт можно вызвать с помощью **npm start**.
- **test**: цель этого скрипта заключается в том, чтобы запускать ваши тесты: одно из удобств такого запуска тестов заключается в том, что в этом случае mocha не нужно устанавливать глобально, так как NPM будет искать её в папке node_modules/.bin, а mocha будет размещена там же. Запуск тестов может быть вызван с помощью **npm test**.

- **your-custom-script**: всё, что вы захотите (вы можете выбрать любое имя). Его можно вызвать с помощью **npm run your-custom-script** - не забывайте про run в этом случае.

Ограничение области видимости и приватные пакеты

Первоначально у NPM было глобальное пространство имен для названий модулей, и с более чем 250 000 модулями в реестре большинство простых имен уже заняты. Кроме того, глобальное пространство имен содержит только общедоступные модули.

В NPM урегулировали эту проблему внедрением пакетов с ограниченной областью видимости (scoped packages), они имеют следующий шаблон именования:

@myorg/mypackage

Вы можете устанавливать пакеты с ограниченной областью видимости так же, как и раньше:

> npm install @myorg/mypackage --save-dev

Они будут отображаться в вашем package.json следующим образом:

```
"dependencies": {
  "@myorg/mypackage": "^1.0.0"
}
```

Подключения пакетов с ограниченной областью видимости работает должным образом:

> require('@myorg/mypackage')

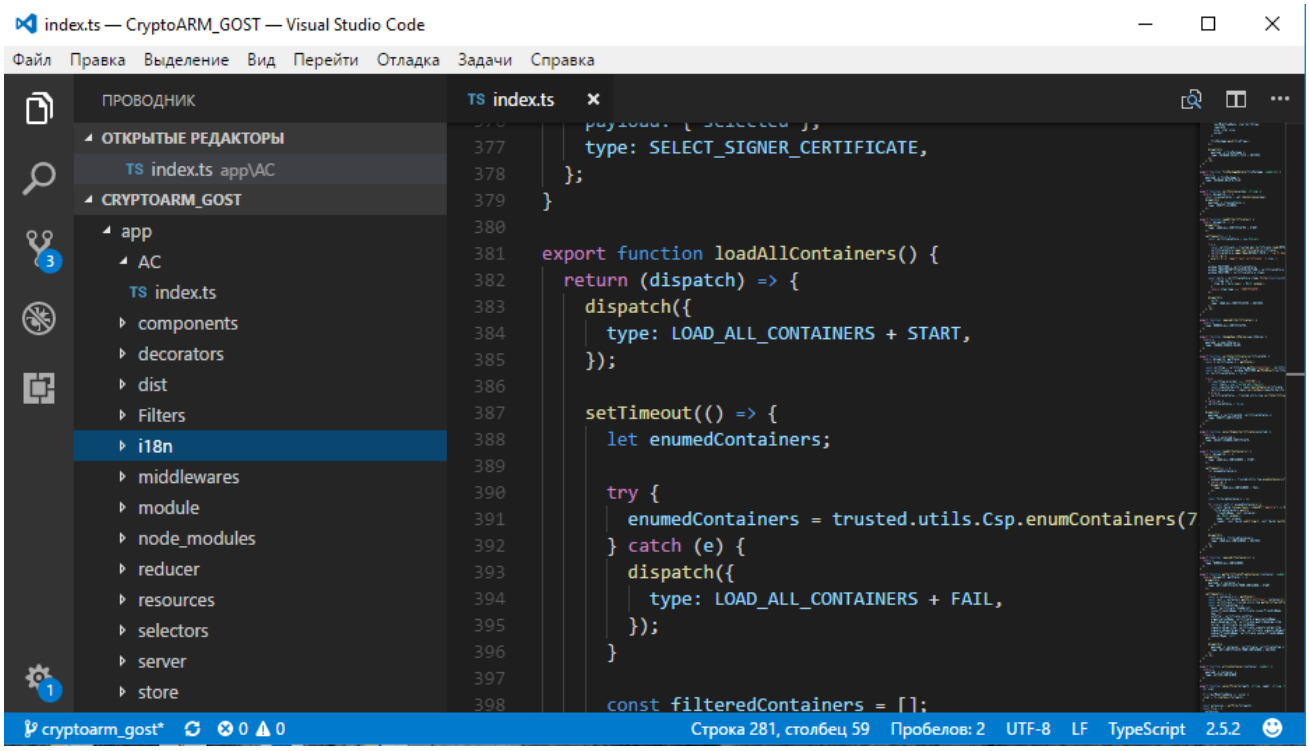
РЕДАКТОР MS VISUAL CODE.

Visual Studio Code (https://code.visualstudio.com/) – популярный бесплатный редактор кода, созданный Microsoft для программистов. VS Code никак не связан с Visual Studio. VS Code работает быстрее редактора Атом, активно развивается и легко расширяется плагинами.

Возможности, которые предоставляет редактор:

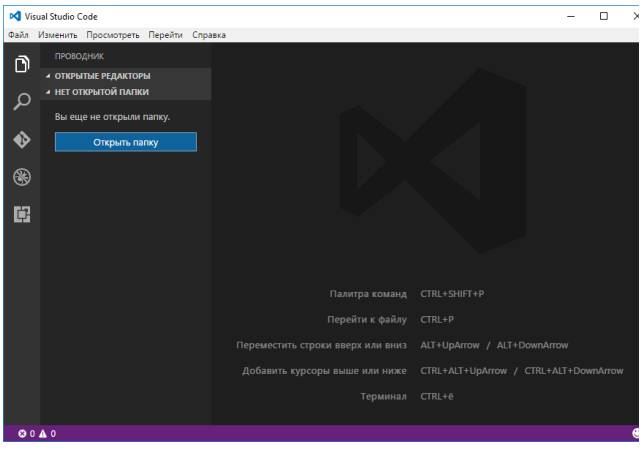
- отладчик кода;
- встроенный терминал;
- удобные инструменты для работы с Git;
- подсветка синтаксиса для множества популярных языков и файловых форматов;
- удобная навигация;
- встроенный предпросмотр Markdown;
- умное автодополнение;
- встроенный пакетный менеджер.

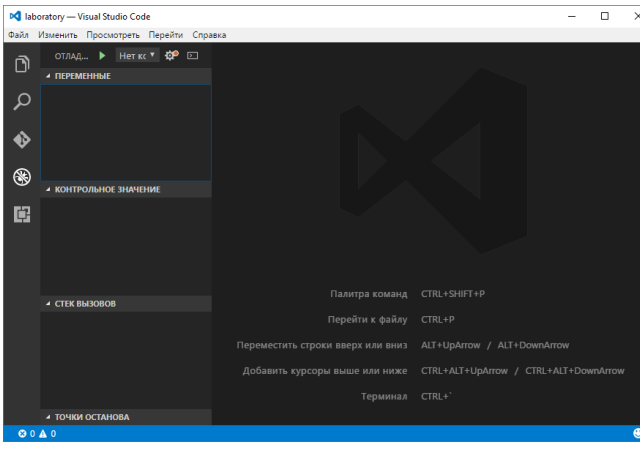
Пакетный менеджер в редакторе используется для установки и удаления пакетов расширений (плагинов). Для удобной разработки на JavaScript для бэкенда и фронтенда иногда возникает необходимость установки нескольких пакетов.



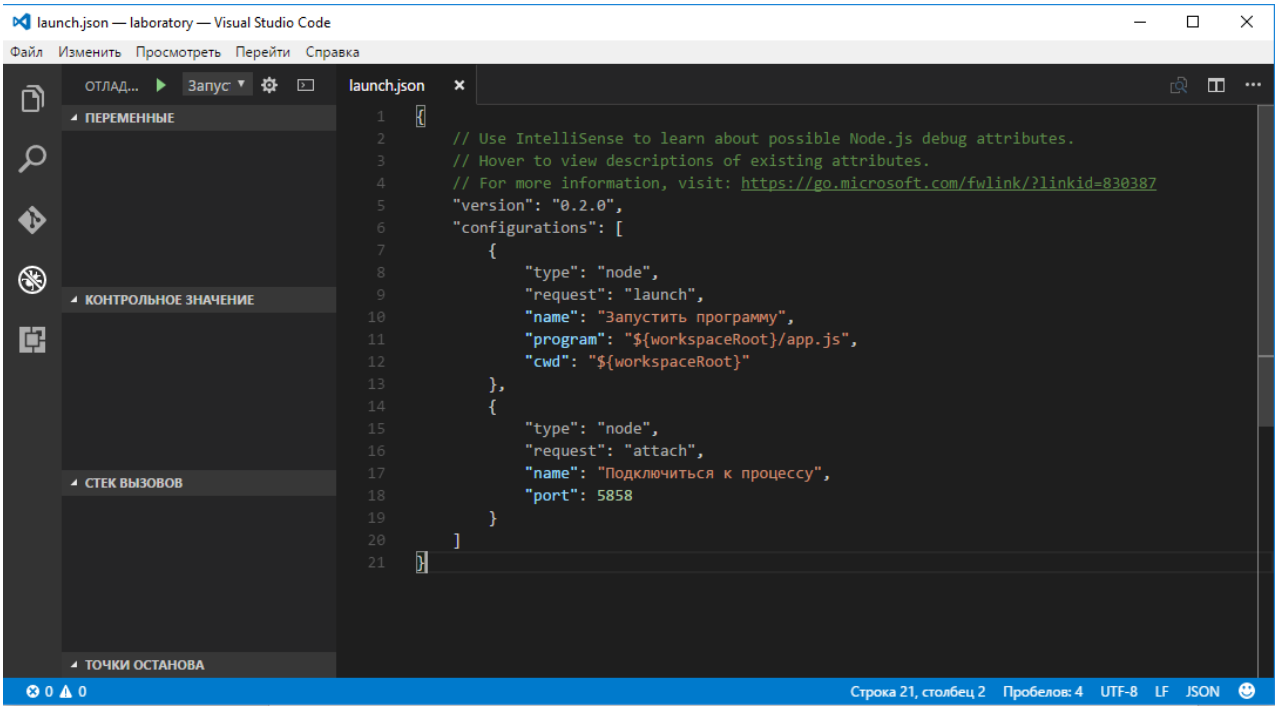
Настройка отладки скриптов VS Code через Node.js

Для того, чтобы вы могли отлаживать программный код в редакторе VS Code, его необходимо связать с установленной библиотекой Node.js, через которую и будет осуществляться запуск приложений. После установки MS visual Code создайте каталог, в котором будут находится ваши скрипты, например, каталог /laboratory. Добавьте тестовый скрипт app.js с командой console.log(“Hello world!”). Запустите редактор и откройте созданную папку в интерфейсе. Затем перейдите в режим отладки кода через боковое меню (сочетание клавиш CTRL+SHIFT+D). Должно открыться окно, изображенное на рисунке ниже.

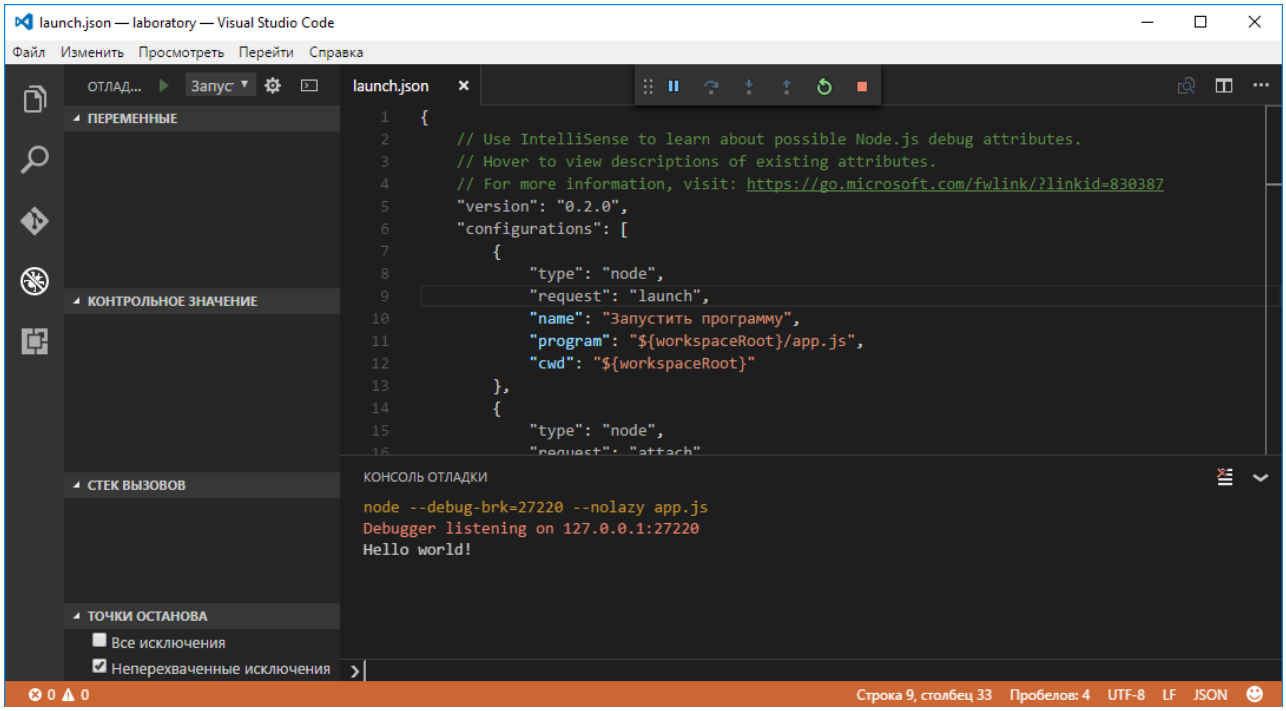




Нажмите на кнопку  настройки файла конфигурации launch.json и выберите среду отладки Node.js. В редакторе должен открыться файл launch.json, который физически будет расположен в выбранном каталоге /laboratory.



Чтобы проверить работу IDE в режиме отладки нажмите на кнопку  (клавиша F5) и на консоли должен появиться результат выполнения скрипта app.js, как показано на рисунке ниже.



Консоль можно показывать и скрывать кнопкой  . Для того, чтобы отлаживать собственный набор скриптов вам необходимо переименовать тестовый скрипт app.js или добавить дополнительные секции в файл конфигурации launch.json.

На этом установку и настройку инструментария можно считать законченным и приступать к изучению теоретического материала и выполнению заданий лабораторной работы.

IDE JETBRAINS WEBSTORM.

WebStorm – среда для разработки на JavaScript, которая подходит для client-side-разработки, создания приложений на Node.js и мобильных приложений на React Native.

Официальный сайт разработчика -

Главное достоинство WebStorm – это удобный и умный редактор для JavaScript, HTML и CSS, который также поддерживает TypeScript, CoffeeScript, Dart, Less, Sass и Stylus и фреймворки, например, Angular, React и Vue.js.

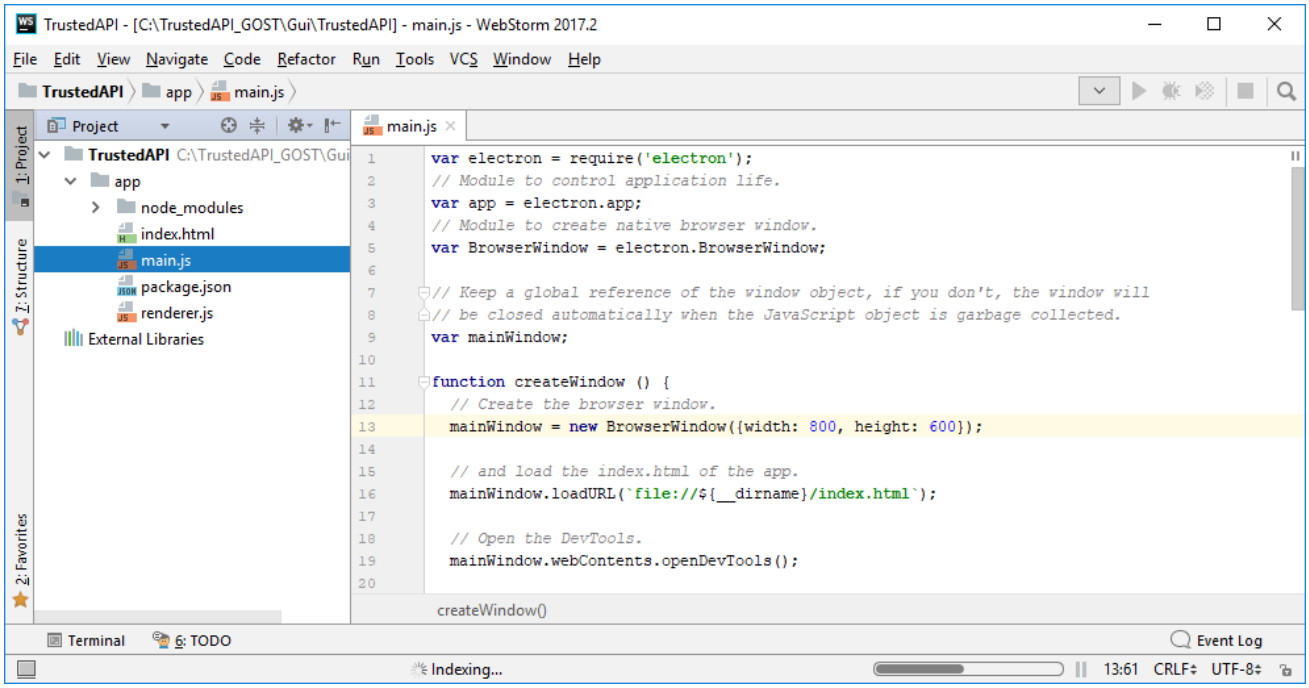
WebStorm, как и другие IDE на платформе IntelliJ IDEA, делает разработку проще и удобней. WebStorm обеспечивая подсветку и автодополнение кода, проверяет его на ошибки, помогает быстро выполнять навигацию по проекту и безопасно вносить изменения с помощью рефакторингов. В WebStorm есть инструменты для отладки кода и интеграция с системами управления версиями.

WebStorm по-настоящему понимает структуру вашего проекта и код, обнаруживает возможные проблемы еще до того, как вы открыли проект в браузере, и предлагает их решение.

Встроенные в IDE инструменты для тестирования помогут в разработке и сделают ее удобней и продуктивней.

Ключевые возможности среды разработки:

- отладка client-side-, Node.js- и React Native-приложений прямо в WebStorm;
- помощь при работе с приложениями на Angular, React, Vue.js, Electron и Meteor;
- интеграция с системами управления версиями Git, GitHub, Subversion, Perforce и Mercurial;
- интеграция с системами отслеживания ошибок (ESLint, JSHint, JSLint, TSLint и Stylelint).



Live Edit позволяет мгновенно видеть результаты ваших изменений в CSS, HTML и JavaScript файлах проекта в браузере без перезагрузки страницы. Live Edit доступен в режиме отладки JavaScript и работает в браузере Google Chrome с предустановленным плагином JetBrains IDE Support.

WebStorm поддерживает современные языки, компилируемые в JavaScript, такие как TypeScript, CoffeeScript и новую версию JavaScript - ECMAScript 2017, предлагая автодополнение кода, проверку ошибок, рефакторинг и отладку.

WebStorm позволяет эффективно разрабатывать приложения на Node.js и поддерживает полноценную отладку Node.js приложений. Новое приложение можно создать, используя шаблон Node.js Express, а необходимые модули легко установить через npm, не используя командную строку.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

1. Пауэрс Ш. Изучаем Node. Переходим на сторону сервера. 2-е изд. Изд. Питер, 2017 – 304с.
2. Давид Хэррон. Node.js Разработка серверных веб-приложений на JavaScript. –М.: ДМК Пресс, 2012 – 144с.
3. Сухов.К.К. Node.js. Путеводитель по технологии. –М.: ДМК Пресс, 2015. - 416 с.
4. Кантелон М., Хартер, М. Головайчук TJ, Рейлих Н. Node.js в действии. – СПб.: Питер, 2015. – 448 с.
5. Аквино К., Ганди Т. Front-end. Клиентская разработка для профессионалов. Node.js, ES6, REST. Изд. Питер, 2017. – 512с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Каким инструментарием из числа online-сервисов можно воспользоваться для исполнения кода с пользовательскими диалогами типа alert или confirm?
2. Что такое библиотека Node.js? Какие возможности она предоставляет?
3. Как переключиться в REPL режим и выполнить программный код JavaScript?
4. Основные команды REPL режима. Как выполнить многострочный код?
5. Как можно создать приложение для Node.js? Его основные составляющие.
6. Как подключить внутренние/внешние модули Node.js в ваш проект?
7. Что такое NPM? Основные команды для работы с npm.
8. Основные возможности редактора VS Code. Как его связать с Node.js для отладки кода?
9. Основные возможности среды разработки WebStorm.